```
//
================================================
=================
// MD RAKIB ISLAM — PORTFOLIO SCRIPT
//
================================================
=================

// ---------- Configuration (editable data)
----------
const games = [
  {
    name: "Free Fire",
    icon: "🔥",
    category: "Battle Royale",
    description: "A fast-paced competitive
battle royale experience."
  },
  {
    name: "PUBG",
    icon: "🎯",
    category: "Battle Royale",
    description: "A competitive tactical
battle royale experience."
  },
  {
    name: "Clash of Clans",
    icon: "🏰",
    category: "Strategy",
    description: "A strategy and base-
```

```
building game focused on planning and
progression."
  },
  {
    name: "E-Football",
    icon: "⚽",
    category: "Football / Sports",
    description: "A football gaming
experience focused on matches, tactics and
team play."
  }
];

const galleryImages = [
  { src: "assets/images/rakib-01.jpg", alt:
"MD Rakib Islam holding lotus flowers at a
pond", layout: "g-large" },
  { src: "assets/images/rakib-02.jpg", alt:
"MD Rakib Islam portrait near the lotus
pond", layout: "" },
  { src: "assets/images/rakib-03.jpg", alt:
"MD Rakib Islam standing in water at
sunset", layout: "" },
  { src: "assets/images/rakib-04.jpg", alt:
"MD Rakib Islam sitting near a decorated
heart display", layout: "g-wide" },
  { src: "assets/images/rakib-05.jpg", alt:
"MD Rakib Islam with a friend at a cafe",
layout: "" },
  { src: "assets/images/rakib-06.jpg", alt:
"MD Rakib Islam with friends at night",
layout: "" }
];
```

```
// Optional data placeholders — not
displayed until real info is supplied
const optionalGamingData = {
  freeFireUID: null,
  freeFireRank: null,
  pubgUID: null,
  pubgRank: null,
  cocPlayerTag: null,
  townHall: null,
  clanName: null,
  eFootballDivision: null,
  favoriteTeam: null,
  favoritePlayer: null,
  gamingSetup: null,
  device: null,
  achievements: null,
  tournamentHistory: null,
  youtube: null,
  tiktok: null,
  telegram: null,
  discord: null
};

const typingWords = ["Gamer", "Student",
"Gaming Enthusiast", "Humanities Student",
"Future Creator"];

// ---------- Navigation ----------
function initNavbar() {
  const navbar =
document.getElementById("navbar");
  const onScroll = () => {
    navbar.classList.toggle("scrolled",
window.scrollY > 30);
```

```
  };
  window.addEventListener("scroll",
onScroll, { passive: true });
  onScroll();
}

function initSmoothScroll() {

document.querySelectorAll('a[href^="#"]').f
orEach(link => {
    link.addEventListener("click", e => {
      const id = link.getAttribute("href");
      if (id.length > 1) {
        const target =
document.querySelector(id);
        if (target) {
          e.preventDefault();
          target.scrollIntoView({ behavior:
"smooth", block: "start" });
        }
      }
    });
  });
}

// ---------- Mobile Menu ----------
function initMobileMenu() {
  const hamburger =
document.getElementById("hamburger");
  const mobileMenu =
document.getElementById("mobileMenu");

  const closeMenu = () => {
    hamburger.classList.remove("open");
```

```
    hamburger.setAttribute("aria-expanded",
"false");
    mobileMenu.classList.remove("open");
  };

  hamburger.addEventListener("click", () =>
{
    const isOpen =
mobileMenu.classList.toggle("open");
    hamburger.classList.toggle("open",
isOpen);
    hamburger.setAttribute("aria-expanded",
String(isOpen));
  });


mobileMenu.querySelectorAll("a").forEach(li
nk => {
    link.addEventListener("click",
closeMenu);
  });
}

// ---------- Active Section Highlighting -
---------
function initActiveSection() {
  const sections =
document.querySelectorAll("main
section[id]");
  const navLinks =
document.querySelectorAll(".nav-link,
.mobile-links a");

  const setActive = id => {
```

```
    navLinks.forEach(link => {
      link.classList.toggle("active",
link.dataset.section === id);
    });
  };

  const observer = new
IntersectionObserver(
    entries => {
      entries.forEach(entry => {
        if (entry.isIntersecting) {
          setActive(entry.target.id);
        }
      });
    },
    { rootMargin: "-45% 0px -50% 0px",
threshold: 0 }
  );

  sections.forEach(section =>
observer.observe(section));
}

// ---------- Typing Animation ----------
function initTypingAnimation() {
  const el =
document.getElementById("typingText");
  if (!el) return;

  const reduceMotion = window.matchMedia("
(prefers-reduced-motion: reduce)").matches;
  if (reduceMotion) {
    el.textContent = typingWords[0];
    return;
```

```
  }

  let wordIndex = 0;
  let charIndex = 0;
  let deleting = false;

  const TYPE_SPEED = 90;
  const DELETE_SPEED = 45;
  const HOLD_TIME = 1400;

  function tick() {
    const word = typingWords[wordIndex];

    if (!deleting) {
      charIndex++;
      el.textContent = word.slice(0,
charIndex);
      if (charIndex === word.length) {
        deleting = true;
        setTimeout(tick, HOLD_TIME);
        return;
      }
      setTimeout(tick, TYPE_SPEED);
    } else {
      charIndex--;
      el.textContent = word.slice(0,
charIndex);
      if (charIndex === 0) {
        deleting = false;
        wordIndex = (wordIndex + 1) %
typingWords.length;
        setTimeout(tick, 300);
        return;
      }
```

```
      setTimeout(tick, DELETE_SPEED);
    }
  }

  tick();
}

// ---------- Scroll Reveal ----------
function initScrollReveal() {
  const items =
document.querySelectorAll(".reveal");
  const observer = new
IntersectionObserver(
    entries => {
      entries.forEach(entry => {
        if (entry.isIntersecting) {
          entry.target.classList.add("in-
view");
          observer.unobserve(entry.target);
        }
      });
    },
    { threshold: 0.12 }
  );
  items.forEach(item =>
observer.observe(item));
}

// ---------- Gaming Data (render games) --
--------
function renderGames() {
  const grid =
document.getElementById("gamesGrid");
  if (!grid) return;
```

```
  grid.innerHTML = games
    .map(
      (game, i) => `
    <div class="game-card reveal">
      <span class="game-card-num">GAME
${String(i + 1).padStart(2, "0")}</span>
      <span class="game-card-icon" aria-
hidden="true">${game.icon}</span>
      <h4>${game.name}</h4>
      <span class="game-card-
cat">${game.category}</span>
      <p>${game.description}</p>
    </div>
  `
    )
    .join("");

  // Re-observe newly injected reveal items

grid.querySelectorAll(".reveal").forEach(el
=> {
    revealObserverRef &&
revealObserverRef.observe(el);
  });
}

let revealObserverRef = null;
function initScrollRevealWithRef() {
  const observer = new
IntersectionObserver(
    entries => {
      entries.forEach(entry => {
        if (entry.isIntersecting) {
```

```
          entry.target.classList.add("in-
view");
          observer.unobserve(entry.target);
        }
      });
    },
    { threshold: 0.12 }
  );
  revealObserverRef = observer;

document.querySelectorAll(".reveal").forEac
h(el => observer.observe(el));
}

// ---------- Gallery ----------
function renderGallery() {
  const grid =
document.getElementById("galleryGrid");
  if (!grid) return;

  grid.innerHTML = galleryImages
    .map(
      (img, i) => `
    <div class="gallery-item reveal
${img.layout}" data-index="${i}">
      <img src="${img.src}"
alt="${img.alt}" loading="lazy">
      <div class="gallery-overlay">
<span>View</span></div>
    </div>
  `
    )
    .join("");
```

```
  grid.querySelectorAll(".gallery-
item").forEach(item => {
    item.addEventListener("click", () =>
openLightbox(Number(item.dataset.index)));
  });


grid.querySelectorAll(".reveal").forEach(el
=> {
    revealObserverRef &&
revealObserverRef.observe(el);
  });
}

// ---------- Lightbox ----------
let currentImageIndex = 0;

function initLightbox() {
  const lightbox =
document.getElementById("lightbox");
  const img =
document.getElementById("lightboxImg");
  const closeBtn =
document.getElementById("lightboxClose");
  const prevBtn =
document.getElementById("lightboxPrev");
  const nextBtn =
document.getElementById("lightboxNext");

  closeBtn.addEventListener("click",
closeLightbox);
  prevBtn.addEventListener("click", () =>
showImage(currentImageIndex - 1));
  nextBtn.addEventListener("click", () =>
```

```
showImage(currentImageIndex + 1));

  lightbox.addEventListener("click", e => {
    if (e.target === lightbox)
closeLightbox();
  });

  document.addEventListener("keydown", e =>
{
    if
(!lightbox.classList.contains("open"))
return;
    if (e.key === "Escape")
closeLightbox();
    if (e.key === "ArrowLeft")
showImage(currentImageIndex - 1);
    if (e.key === "ArrowRight")
showImage(currentImageIndex + 1);
  });

  // Touch swipe
  let touchStartX = 0;
  lightbox.addEventListener(
    "touchstart",
    e => {
      touchStartX =
e.changedTouches[0].screenX;
    },
    { passive: true }
  );
  lightbox.addEventListener(
    "touchend",
    e => {
      const touchEndX =
```

```
e.changedTouches[0].screenX;
      const diff = touchEndX - touchStartX;
      if (Math.abs(diff) > 50) {
        diff > 0 ?
showImage(currentImageIndex - 1) :
showImage(currentImageIndex + 1);
      }
    },
    { passive: true }
  );
}

function openLightbox(index) {
  document.body.style.overflow = "hidden";

document.getElementById("lightbox").classLi
st.add("open");
  showImage(index);
}

function closeLightbox() {
  document.body.style.overflow = "";

document.getElementById("lightbox").classLi
st.remove("open");

document.getElementById("lightboxImg").clas
sList.remove("show");
}

function showImage(index) {
  const total = galleryImages.length;
  currentImageIndex = (index + total) %
total;
```

```
  const data =
galleryImages[currentImageIndex];
  const img =
document.getElementById("lightboxImg");

  img.classList.remove("show");
  const temp = new Image();
  temp.src = data.src;
  temp.onload = () => {
    img.src = data.src;
    img.alt = data.alt;
    requestAnimationFrame(() =>
img.classList.add("show"));
  };


document.getElementById("lightboxCounter").
textContent =
    String(currentImageIndex +
1).padStart(2, "0") + " / " +
String(total).padStart(2, "0");
}

// ---------- Contact Menu (floating
button) ----------
function initFloatingContact() {
  const btn =
document.getElementById("floatingBtn");
  const menu =
document.getElementById("floatingMenu");
  const wrapper =
document.getElementById("floatingContact");

  btn.addEventListener("click", e => {
```

```
    e.stopPropagation();
    const isOpen =
menu.classList.toggle("open");
    btn.setAttribute("aria-expanded",
String(isOpen));
  });

  document.addEventListener("click", e => {
    if (!wrapper.contains(e.target)) {
      menu.classList.remove("open");
      btn.setAttribute("aria-expanded",
"false");
    }
  });
}

// ---------- Back To Top ----------
function initBackToTop() {
  const btn =
document.getElementById("backToTop");

  window.addEventListener(
    "scroll",
    () => {
      btn.classList.toggle("show",
window.scrollY > 600);
    },
    { passive: true }
  );

  btn.addEventListener("click", () => {
    window.scrollTo({ top: 0, behavior:
"smooth" });
  });
```

```
}

// ---------- Initialization ----------
document.addEventListener("DOMContentLoaded
", () => {
  initNavbar();
  initSmoothScroll();
  initMobileMenu();
  initActiveSection();
  initTypingAnimation();
  initScrollRevealWithRef();
  renderGames();
  renderGallery();
  initLightbox();
  initFloatingContact();
  initBackToTop();
});
```